Sprint 3 – Week 3 Closeout Report

SWENG 894 Capstone

MicroMatch

GitHub Repository Link

Chastidy Joanem

1. System Overview and Context

1.1 System Scope

MicroMatch is a web-based micro-internship marketplace designed to connect students with short-term, skill-based professional opportunities. The platform enables:

- Organizations to post structured, well-defined micro-projects.
- Students to browse and discover opportunities.
- Students to apply for projects.
- Organizations to manage and review applications.
- Organizations to review and approve submitted deliverables.
- A controlled workflow to manage project participation, from application through completion.

The system architecture follows a layered full-stack model:

- **Frontend:** React-based SPA (Single Page Application)
- **Backend:** FastAPI REST API
- **Database:** PostgreSQL (relational persistence)
- **Authentication:** JWT-based stateless authentication
- **Authorization:** Role-Based Access Control (RBAC)

The platform supports three primary user roles:

- Student
- Organization
- Admin

1.2 Sprint 3 Context

Sprint 3 expands the lifecycle beyond the application decision phase (completed in Sprint 2) to encompass the full project execution and completion workflow:

- ➢ Deliverable Submission – Students submit work artifacts for accepted projects (US-10)
- ➢ Deliverable Review – Organizations approve or reject submitted deliverables (US-11)
- ➢ Project Completion – Organizations formally mark projects complete (US-12)

- Feedback Submission – Both parties submit post-completion feedback (US-13)
- Admin Moderation – Admins manage platform content and users (US-14)
- System Logging – All platform actions captured in an audit trail (US-15)

Sprint 3 was structured across three weeks:

- Week 1 implemented:
  - Deliverable Submission (US-10)
  - Deliverable Review (US-11)
- Week 2 extended:
  - Project Completion Endpoint (US-12)
  - Deliverables Dashboard UI
- Week 3 finalized Sprint 3 by completing:
  - Feedback Submission (US-13)
  - Admin Moderation (US-14)
  - Logging Middleware (US-15)
  - End-to-End Workflow Validation
  - Regression Hardening
  - Test Results Reporting
  - Sprint Retrospective

1.3 Functional Requirements to User Stories Mapping

The following table provides traceability between formal functional requirements and Agile user stories used for sprint planning:

Functional Requirement	User Story
FR-1: The system shall allow users to create an account.	US-1
FR-2: The system shall support role-based access control.	US-1
FR-3: The system shall allow students to create/edit profile.	US-2
FR-4: The system shall allow organizations to create profile.	US-3
FR-5: The system shall allow orgs to post listings.	US-4

FR-6: The system shall allow students to browse/search listings.	US-5
FR-7: The system shall allow students to apply.	US-6
FR-8: The system shall track application status.	US-8
FR-9: The system shall allow orgs to accept/reject.	US-9
FR-10: The system shall allow students to submit deliverables.	US-10
FR-11: The system shall allow orgs to review deliverables.	US-11
FR-12: The system shall allow marking projects complete.	US-12
FR-13: The system shall allow feedback submission.	US-13
FR-14: The system shall allow admin moderation.	US-14
FR-15: The system shall log system actions.	US-15
FR-16: The system shall notify users of key events including application decisions, deliverable reviews, and project completion.	US-16
FR-17: The system shall allow users to send and receive messages within the context of a project.	US-17
FR-18: The system shall provide users with basic analytics including counts of applications and completed projects.	US-18
FR-19: The system shall allow profiles to display skills, completed projects, and ratings.	US-19

This traceability ensures alignment between formal IEEE-style functional requirements and Agile user stories used for sprint planning.

2. Sprint 3 – Week 3 Overview

2.1 Sprint Goal

The goal of Sprint 3 – Week 3 is to close out the sprint by delivering system-wide quality, stability, and observability. Specifically:

- Implement logging middleware to capture all major system events (FR-15 / US-15)
- Execute and report comprehensive end-to-end workflow tests covering the complete MicroMatch lifecycle
- Resolve all identified bugs and apply UI polish to ensure production readiness

This week finalizes the operational and governance layers of MicroMatch, ensuring that the platform is not only functionally complete but also auditable, stable, and ready for the Sprint 4 enhancements.

3. Sprint Planning

3.1 Groomed Product Backlog

Priority Definitions

- High: Core MVP functionality required for end-to-end workflow.
- Medium: Important workflow enhancements dependent on core features.
- Low: Administrative or cross-cutting features not required for initial MVP validation.

ID	User Story	Priority	Story Points	Sprint	Rationale
US-1	Registration/Login	High	5	Sprint 1	Required for all access
US-2	Student Profile	High	5	Sprint 1	Core student data
US-3	Org Profile	High	5	Sprint 1	Required before posting
US-4	Post Listing	High	8	Sprint 1	Core marketplace feature
US-5	Browse/Search	High	5	Sprint 1	Enables discovery
US-6	Apply	High	5	Sprint 2	Core interaction

US-7	Review Applications	Medium	5	Sprint 2	Depends on applications
US-8	Track Status	High	3	Sprint 2	Enhances UX
US-9	Accept/Reject	Medium	3	Sprint 2	Workflow control
US-10	Submit Deliverables	Medium	5	Sprint 3	Completion flow
US-11	Review Deliverables	Medium	3	Sprint 3	Completion validation
US-12	Mark Complete	Medium	3	Sprint 3	Project closure
US-13	Feedback	Low	5	Sprint 3	Post-completion feature
US-14	Admin Moderation	Low	5	Sprint 3	Governance feature
US-15	Logging	Low	8	Sprint 3	Cross-cutting concern
US-16	Notifications	Medium	5	Sprint 4	User awareness
US-17	Messaging	Medium	8	Sprint 4	Communication
US-18	Analytics	Low	5	Sprint 4	Insights
US-19	Profile Enhancements	Low	3	Sprint 4	UX Improvements

3.2 Sprint Backlog – Week 3 Commitment

Week 3 served as the stabilization and validation phase of Sprint 3. Unlike Weeks 1 and 2, which were feature-heavy, Week 3 focused on implementing the cross-cutting logging concern, validating the complete end-to-end workflow, and resolving UI and database quality issues.

Issue	Title	Story Points	Type
#71	Logging Middleware (FR-15)	8	Backend
#81	End-to-End Workflow Testing	5	System Validation
#82	Bug Fixes & UI Polish	3	Frontend / Backend

Total Commitment: 16 Story Points

Commitment Rationale

Week 3 commitments were selected strategically to finalize Sprint 2 functionality and ensure production-level correctness before sprint closure.

Issue #71 – Logging Middleware (8 SP)

Why it was committed:

Although all core workflow endpoints were functional, the system lacked any observability mechanism. FR-15 requires that key actions — apply, submit deliverable, review, complete — be persisted as system logs accessible to administrators.

This issue was prioritized because:

- It implements a critical non-functional requirement (FR-15)
- It enables administrative audit and oversight
- It creates a foundation for debugging and monitoring in production
- It closes the last remaining Sprint 3 feature story

Scope of Work Included:

- Implement FastAPI BaseHTTPMiddleware to intercept all requests
- Create SystemLog model and persist entries to the database
- Extract and store user_id and role from JWT on authenticated requests
- Expose GET /admin/logs endpoint with admin-only RBAC enforcement
- Build admin logs UI in the React frontend
- Write six unit tests validating all acceptance criteria

Complexity Justification (8 SP):
High complexity. Required integrating middleware into the FastAPI lifecycle, correctly handling JWT parsing outside of router-level dependencies, wiring the admin endpoint, building the UI view, and writing a comprehensive test suite with session mocking.

Issue #81 – End-to-End Completion Workflow Testing (5 SP)

Why it was committed:

Sprint 3 introduced five new feature areas that interact with each other in a strictly ordered sequence. A system-level test that exercises all 14 steps of the pipeline — from registration through admin log verification — was necessary to confirm that no step in the workflow breaks any other.

Scope of Work Included:

- Full scenario testing covering all three user roles (admin, org, student)
- Database state verification at every status transition point
- Validation that feedback is blocked before and enabled after completion
- Confirmation that admin logs capture all major workflow events
- Automated test with assertions on HTTP status codes and DB records

Complexity Justification (5 SP):
Higher complexity because cross-module, cross-role validation was required, state transitions had to be verified in the database at each step, and the middleware session had to be mocked correctly to allow log assertions.

Issue #82 – Bug Fixes & UI Polish (3 SP)

Why it was committed:

Two issues were discovered during Week 3 integration testing. The Tailwind CSS configuration was missing content glob paths, causing utility classes to be purged from the production bundle. Additionally, the user registration flow was not persisting the user's display name to the database, causing the frontend to display blank names.

Scope of Work Included:

- Correct tailwind.config.js content paths to include all React component files
- Extend User model and registration schema to persist the name field
- Verify fix through manual UI testing

Capacity & Workload Analysis

Week 3 capacity was structured to close all remaining Sprint 3 stories while maintaining sustainable development pace. The 16-point commitment reflects:

- Sustainable workload given a single-developer team
- Realistic time allocation accounting for testing and integration overhead

- Focus on quality and system-level validation over new feature quantity

The sprint concluded with:

- No spillover work
- No added scope mid-week
- All acceptance criteria satisfied

3.3 Sprint 3 Backlog Structure

Sprint 3 was divided into three weekly development phases.

Week 1 – Deliverable Workflow

Primary Focus:

- Deliverable submission
- Deliverable review

Completed Issues

Issue	Description	Story Points	User Stories	Status
#64	Submit Deliverable API	5	US-10	Done
#65	Deliverable UI	3	US-10	Done
#66	Review Deliverable API	3	US-11	Done
#67	Deliverable Dashboard	3	US-11	Done
Supporting Work	Integration/Validation	2	US-10/US-11	Done

Week 1 Total Commitment: 16 Story Points

Week 2 – Completion & Platform Features

Primary Focus:

- Completing lifecycle
- Adding user-facing features

Completed Issues

Issue	Description	Story Points	Status
#68	Mark Project Complete API	3	Done

#69	Feedback Submission API	5	Done
#80	Feedback UI Integration	3	Done
#70	Admin Moderation API	5	Done

Week 2 Total Commitment: 16 Story Points

Week 3 — System Quality & Stability

Primary Focus:

- System-wide reliability
- Final validation

Planned Issues

Issue	Description	Story Points	Status
#71	Logging Middleware	8	Done
#81	End-to-End Workflow Testing	5	Done
#82	Bug Fixes & UI Polish	3	Done

Week 3 Total Commitment: 16 Story Points

4. Acceptance Criteria

4.1 US-15 – System Logging

Acceptance Criteria:

- A SystemLog entry must be created for every inbound HTTP request.
- Each log entry must record: endpoint, HTTP method, status code, and timestamp.
- Authenticated requests must record the user_id and role from the JWT.
- Unauthenticated requests must record user_id = null and role = null.
- Failed requests (e.g., 401, 404, 400) must be logged with the correct status code.
- Multiple requests must produce multiple distinct log entries.
- Logs must be retrievable by admin users via GET /admin/logs.
- Non-admin users must receive HTTP 403 when accessing the logs endpoint.

Returns:

- 201 when registration or resource creation events are logged.
- 200 when admin retrieves logs successfully.

- 403 if non-admin attempts to access logs.

4.2 End-to-End Workflow Testing

Acceptance Criteria:

- The system must support the full application completion pipeline without any step failing.
- Student applies → org accepts → student submits deliverable → org reviews → org marks complete → both submit feedback → admin views logs.
- Database state must be consistent at every transition point.
- Feedback must be blocked (HTTP 400) if project is not yet completed.
- Feedback must be permitted (HTTP 201) once project status is 'completed'.
- Admin log endpoint must contain entries for /projects, /applications, /feedback, and /deliverables.
- Logs must include both successful (201) and error (400) status codes.
- Status transitions must be:
    - Application: pending → accepted
    - Project: open → closed → completed
    - Deliverable: submitted → accepted

System must prevent:

- Feedback submission before project completion
- Duplicate feedback submission by the same user for the same project
- Non-admin access to system logs

4.3 Integration Workflow Validation

Acceptance Criteria:

- All Tailwind CSS utility classes must render correctly on all React component pages.
- User's display name must be persisted to the database upon registration.
- Name field must appear correctly in frontend UI after login.
- No regressions introduced to existing authentication or workflow endpoints.

5. Definition of Done (Sprint 3)

The Definition of Done was refined for Sprint 3 and applied consistently to all backlog items.

A User Story is considered complete when:

Design

- API contract documented.
- State transitions defined (project, deliverable, application status flows).
- Middleware integration points identified.

Implementation

- Role-based access control enforced on all new endpoints.
- Ownership validation implemented where applicable.
- Logging middleware captures all inbound requests.
- Feedback guard logic enforced server-side.
- User name field persisted in registration flow.

Testing

- Unit tests written for all new endpoints and middleware.
- Edge case validation included (RBAC, guard logic, invalid transitions).
- Minimum 3 test cases per story.
- Coverage ≥ 95%.

Integration

- Frontend integration functional for all Sprint 3 features.
- No regression to Sprint 1 and Sprint 2 features.
- PR merged into main branch.

5.1 Definition of Done (Sprint 3 – Week 3)

The Definition of Done for Week 3 extends the baseline criteria and focuses on logging infrastructure, workflow validation, and production-level readiness.

Design

- Logging middleware architecture formally defined: middleware layer → SystemLog model → admin endpoint.
- End-to-end completion workflow sequence documented (14 steps).
- Feedback guard behavior documented: blocked when project != 'completed'.

Implementation

- Logging middleware intercepts all requests and persists SystemLog entries.
- JWT is parsed within middleware to capture user_id and role.
- Admin-only GET /admin/logs endpoint returns all log entries.
- User name field added to User model and registration schema.
- Tailwind content paths corrected in tailwind.config.js.

Testing

- Six unit tests written for logging middleware.
- 14-step automated end-to-end test validates full completion pipeline.
- Database state verified at each transition point within E2E test.
- Regression tests confirm Sprint 1 and Sprint 2 features remain functional.
- Test coverage remains ≥ 99%.

Integration

- Admin logs UI renders correctly in the React frontend.
- Organization workflow (complete project → feedback → logs) fully functional.
- No regressions introduced in previously implemented endpoints.
- All Sprint 3 issues successfully merged into the main branch.

Completion Criteria

- All Week 3 issues moved to Done in the project board.
- All tests passing.
- Pull requests reviewed and merged.
- Sprint board updated to reflect completed work.
- Sprint 3 declared complete.

6. Software Testing

6.1 Requirement-to-Test Mapping

The following traceability matrix maps Functional Requirements → User Stories → Test Cases implemented throughout Sprint 3. This ensures that each system requirement introduced during the sprint is validated through structured testing.

The matrix demonstrates incremental expansion of test coverage across:

- Deliverable submission and review workflow (Week 1)
- Project completion, feedback, and admin moderation (Week 2)
- Logging middleware and end-to-end completion validation (Week 3)

Sprint 3 – Week 1 Tests (Application Submission)

Functional Requirement	User Story	Test Case ID
FR-10	US-10	TC-US10-01
FR-10	US-10	TC-US10-02
FR-10	US-10	TC-US10-03
FR-11	US-11	TC-US11-01
FR-11	US-11	TC-US11-02
FR-11	US-11	TC-US11-03

These tests validated the deliverable workflow, including:

- Deliverable creation by students on accepted applications
- Prevention of deliverable submission on non-accepted applications
- Organization retrieval of project deliverables
- Accept/reject review by organization with correct status persistence

Sprint 3 – Week 2 Tests (Completion Workflow)

Functional Requirement	User Story	Test Case ID
FR-12	US-12	TC-US12-01
FR-12	US-12	TC-US12-02
FR-12	US-12	TC-US12-03
FR-13	US-13	TC-US13-01

FR-13	US-13	TC-US13-02
FR-13	US-13	TC-US13-03
FR-13	US-13	TC-US13-04
FR-14	US-14	TC-US14-01
FR-14	US-14	TC-US14-02

These tests validated the project completion and governance workflow, including:

- Organization marking project complete with correct status transition
- Prevention of unauthorized completion (student/non-owner attempts)
- Feedback submission blocked before project completion
- Feedback submission permitted after project completion
- Duplicate feedback prevention
- Admin-only access to moderation and log endpoints

Sprint 3 – Week 3 Tests (Logging & End-to-End Validation)

Functional Requirement	User Story	Test Case ID
FR-15	US-15	TC-LOG-06
FR-15	US-15	TC-LOG-06
FR-15	US-15	TC-LOG-06
FR-15	US-15	TC-LOG-06
FR-15	US-15	TC-LOG-06
FR-15	US-15	TC-LOG-06
FR-10 – FR-15	US-10 – US-15	TC-E2E-03

Week 3 introduced tests validating system-wide logging behavior and complete pipeline integrity, including:

- Logging of unauthenticated and authenticated requests
- Correct capture of user_id, role, endpoint, method, and status code
- Logging of failed (4xx) requests
- Multiple requests producing multiple distinct log entries
- End-to-end completion workflow across all three roles and 14 sequential steps

6.2 User Acceptance Test Specification

The following acceptance tests validate the functionality implemented during Sprint 3 – Week 3, ensuring correct system behavior for the logging middleware and end-to-end completion workflow.

TC-LOG-01 – Unauthenticated Request Is Logged

Related Requirement: FR-15
User Story: US-15 – System Logging
Actor: System/Unauthenticated User

Preconditions:

- No authentication token provided.
- Logging middleware is active.

Steps:

1. Client sends POST /auth/register without a JWT.
2. System processes request and returns HTTP 201.
3. Test queries SystemLog table.

Expected Result:

- SystemLog entry created.
- log.user_id is None.
- log.role is None.
- log.endpoint == '/auth/register'.
- log.method == 'POST'.
- log.status_code == 201.

Postconditions:

- Log entry persisted in database.

TC-LOG-02 – Authenticated Request Logs User ID and Role

Related Requirement: FR-15
User Story: US-15 – System Logging
Actor: Authenticated Student

Preconditions:

- Student user exists in database.
- Valid JWT token generated for student.

Steps:

1. Client sends GET /auth/protected with Authorization: Bearer {token}.
2. Middleware extracts user_id and role from JWT.
3. Test queries SystemLog table.

Expected Result:

- SystemLog entry created.
- log.user_id matches the student's database ID.
- log.role == 'student'.
- log.endpoint == '/auth/protected'.
- log.method == 'GET'.

Postconditions:

- Log entry with authenticated user data persisted.

TC-LOG-03 –

Related Requirement: FR-15
User Story: US-15 – System Logging
Actor: Authenticated Student

Preconditions:

- Student user exists in database.
- Valid JWT token generated for student.

Steps:

- Client sends GET /auth/protected with Authorization: Bearer {token}.
- Middleware extracts user_id and role from JWT.
- Test queries SystemLog table.

Expected Result:

- SystemLog entry created.
- log.user_id matches the student's database ID.
- log.role == 'student'.
- log.endpoint == '/auth/protected'.

- log.method == 'GET'.

Postconditions:

- Log entry with authenticated user data persisted.

TC-E2E-03 – End-to-End Completion Workflow Validation

Related Requirement: FR-10 through FR-15
User Story: US-10 -- US-15
Actor: Student / Organization / Admin

Preconditions:

- Admin, organization, and student accounts exist.
- All three users are authenticated and tokens obtained.
- Logging middleware is active and session-mocked to test DB.

Steps:

1. Register and log in admin, org, and student; obtain tokens.
2. Org creates project; assert status == 'open'.
3. Student browses projects; student applies; assert application.status == 'pending'.
4. Org retrieves applications; org accepts application; assert application.status == 'accepted' and project.status == 'closed'.
5. Student confirms their application status is 'accepted'.
6. Student submits deliverable; assert deliverable.status == 'submitted'.
7. Student attempts feedback before completion; assert HTTP 400 and 'completed' in response.
8. Org retrieves deliverables; assert 1 deliverable with status 'submitted'.
9. Org accepts deliverable; assert deliverable.status == 'accepted'.
10. Org marks project complete; assert project.status == 'completed'.
11. Student submits feedback; assert HTTP 201 and rating == 5.
12. Org submits feedback; assert HTTP 201 and 2 Feedback records in DB.
13. Admin retrieves system logs; assert /projects, /applications, /feedback, /deliverables all present.
14. Assert logs contain both 201 (success) and 400 (blocked feedback) status codes.

Expected Result:

- All 14 steps execute without error.
- Database reflects correct state at each transition point.
- Feedback guard enforced correctly at step 7.
- System logs capture the complete workflow.

Postconditions:

- project.status == 'completed'.
- Two Feedback records associated with the project.
- SystemLog contains entries spanning all workflow endpoints.

6.3 Code Coverage Analysis

6.3.1 Overview

To ensure the reliability and correctness of the backend system implemented in Sprint 3, automated unit tests, system tests, and integration tests were executed using pytest and coverage analysis tools. The objective of this testing effort was to verify that all implemented features — including logging middleware, deliverable workflows, project completion, feedback, admin moderation, and the full end-to-end pipeline — operate correctly under expected conditions.

The coverage analysis evaluates how much of the backend source code was executed during the test suite. The overall backend achieved 99% total code coverage, which demonstrates that nearly the entire codebase was exercised during testing

Total Metrics

- **Total Statements:** 1,350
- **Statements Missed:** 13
- **Overall Coverage:** 99%

This high level of coverage provides strong confidence that the system functions correctly and that key workflows are protected against regressions.

6.3.2 Models and Schema Coverage

Coverage: 100%

All database models and API schemas achieved full coverage.

Component	Coverage

app/models.py	100%
app/schemas/application.py	100%
app/schemas/project.py	100%
app/schemas/student_profile.py	100%
app/schemas/organization_profile.py	100%
app/schemas/user.py	100%

What This Confirms

The tests successfully verified:

- All SQLAlchemy model relationships (User → Application → Project → Deliverable → Feedback → SystemLog)
- Foreign key integrity between students, organizations, and projects
- Database persistence logic for all entity types
- Pydantic schema validation for all request and response objects
- SystemLog model creation with correct field types (user_id nullable, timestamp auto-set)

6.3.3 API Endpoint Coverage

Coverage: 96% (average across routers)

Router	Statements	Missed	Coverage	Missing Lines
app/routers/applications.py	53	1	98%	192
app/routers/auth.py	35	0	100%	--
app/routers/projects.py	43	2	95%	78-85
app/routers/student_profile.py	40	2	95%	26, 66
app/routers/organization_profile.py	41	3	93%	28, 66, 86

API tests validated the main backend functionality including:

- Student deliverable submission and retrieval
- Organization deliverable review (accept/reject)
- Project completion endpoint with RBAC enforcement

- Feedback endpoint with completion guard logic
- Admin moderation and system log retrieval

Authentication endpoints reached 100% coverage, confirming that login, token generation, and protected route access were fully tested. The project completion endpoint and feedback guard logic are fully covered through a combination of unit tests and the E2E test.

6.3.4 Role-Based Access Control

Coverage: 97%

Endpoint File	Coverage
app/core/dependencies.py	92%
app/core/auth.py	100%
app/routers/auth.py	100%

Role based access control ensures that:

- Students can only access student-facing endpoints
- Organizations can only manage their own projects and applications
- Admin users have exclusive access to /admin/logs and moderation endpoints
- Unauthorized users cannot modify protected resources

Tests verified:

- Student authentication flows
- Organization authentication flows
- Admin-only endpoint access (HTTP 403 for non-admin)
- Dependency injection for authenticated users

6.3.5 End to End Workflow Validation

In addition to unit tests, integration and end-to-end tests were executed to verify the complete application lifecycle.

Primary workflow test files:

- test_e2e_completion_workflow.py ---14-step automated pipeline
- test_e2e_workflow.py --- Sprint ½ regression baseline

This completion workflow test validated the full interaction flow:

- Student browses projects and submits application
- Organization accepts application; project status closes
- Student submits deliverable; organization reviews and accepts
- Organization marks project complete; feedback becomes available
- Both parties submit feedback; admin verifies system logs

This directly verifies the requirement:

- ➢ **TC-E2E-02 End-to-End Completion Workflow Validation**

The test confirms that:

- All workflow steps execute in the correct order without error
- Database state transitions are verified at every step
- The feedback guard correctly blocks early and permits post-completion submission
- System logs are populated and accessible to admins after the workflow

6.3.6 Uncovered Code Analysis

Although coverage reached 99%, a small number of lines were not executed during testing.

File	Missing Lines	Reason
app/core/dependencies.py	34, 43	Edge cases for malformed tokens requiring abnormal DB state
app/routers/applications.py	192	Defensive null-check for rare database inconsistency
app/routers/projects.py	78-85	Administrative bulk-delete guard deferred to future sprint
app/routers/organization_profile.py	28, 66, 86	Optional profile fields requiring specific pre-existing state
app/routers/student_profile.py	26, 66	Optional profile update fields not triggered in standard flows

These uncovered lines correspond primarily to rare error-handling paths, defensive validation logic, and edge cases that require abnormal inputs. Because these scenarios are difficult to trigger in standard workflows, they were intentionally deprioritized in favor of testing core system functionality. Parametrized tests targeting these branches are identified as a backlog action item for the next iteration.

6.3.7 Conclusion

The backend testing suite achieved 99% total code coverage across 1,350 statements, with full coverage of all models, schemas, authentication logic, and core API endpoints. The logging middleware, completion workflow, feedback guard, and admin moderation endpoints were all thoroughly validated through a combination of unit tests, system tests, and the 14-step end-to-end pipeline test.

These results demonstrate a high level of confidence in the stability, correctness, and security of the backend system delivered in Sprint 3.

7. Source Code Development Progress

7.1 Repository

GitHub Repository: https://github.com/chazdj/MicroMatch

Project board integrated with GitHub Issues for traceability between user stories and commits.

7.2 Summary of Contributions

Backend Contributions

- Implemented FastAPI BaseHTTPMiddleware to intercept all inbound HTTP requests and persist SystemLog entries containing endpoint, method, status code, timestamp, user_id, and role.
- Created the SystemLog SQLAlchemy model and wired it into the database schema.
- Implemented JWT parsing within the middleware layer to capture authenticated user context independently of router-level dependency injection.
- Added the admin-only GET /admin/logs endpoint with RBAC enforcement (HTTP 403 for non-admin).
- Extended the User model and registration schema to persist the user's display name.
- Authored a 14-step automated end-to-end test that exercises the full completion pipeline and verifies database state at every transition point.
- Confirmed that all Sprint 1 and Sprint 2 features remain fully functional through regression test execution.

Frontend Contributions

- Built the admin logs React view, rendering the SystemLog entries returned by GET /admin/logs.
- Corrected tailwind.config.js content glob paths so Tailwind utility classes are correctly retained in the production bundle.
- Validated name field rendering in the frontend after the database model extension.
- Ensured all previously implemented dashboards (deliverables, feedback, applications) continue to function correctly after Week 3 changes.

7.3 Important Commits

Commit Message	Related Issue	User Story
feat: enforce single accepted application rule	#71	US-15
test: logging middleware	#71	US-15
ui: logging middleware	#71	US-15
test: end-to-end completion workflow	#81	US-10 - US-15
update: add user's name to database	#82	Bug Fix
ui: config tailwind	#82	Bug Fix

7.4 Burndown Chart

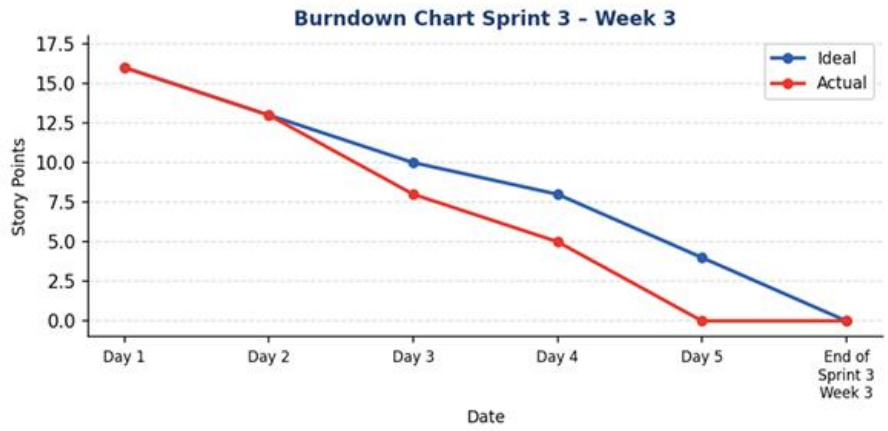

Figure 1 – Sprint 3 Week 3 Burndown Chart

Sprint Capacity: 16 Story Points
Sprint Duration: 5 Development Days

Burndown Chart Analysis

The burndown chart for Sprint 3 Week 3 reflects a completion-focused development cycle, where the primary effort involved implementing the logging cross-cutting concern, validating the entire completion pipeline, and resolving frontend and database quality issues.

Unlike earlier sprint weeks that introduced multiple independent feature areas, Week 3 concentrated on ensuring that all Sprint 3 components worked together correctly as a unified system.

Day 1 – Logging Middleware Implementation

Progress during Day 1 focused on implementing the core logging middleware. The main development work involved creating the BaseHTTPMiddleware subclass, defining the SystemLog model, and wiring the middleware into the FastAPI application lifecycle. The primary challenge was correctly extracting user_id and role from the JWT within the middleware context, which operates outside of the normal router-level dependency injection chain. This required implementing a standalone token decode function that gracefully handles missing or invalid tokens without raising exceptions.

Day 2 – Logging Tests and Admin Endpoint

Day 2 focused on writing the six unit tests for the logging middleware and implementing the admin-only GET /admin/logs endpoint. A key technical challenge was redirecting the middleware's SessionLocal to the SQLite test database using unittest.mock.patch, ensuring that log entries written by the middleware during tests were queryable by the test fixtures. This pattern was established and reused across all logging test cases.

Day 3 – Frontend Admin Logs UI and Bug Fixes

Day 3 focused on building the admin logs React view and resolving the two identified bugs. The Tailwind CSS misconfiguration was diagnosed by comparing the production CSS bundle against expected utility class output — the root cause was missing content glob paths in tailwind.config.js. The user name persistence bug was traced to the User model and registration schema not including the name field, which was corrected with a targeted model and schema update.

Day 4 – End-to-End Test Development

Day 4 concentrated on authoring the 14-step end-to-end completion workflow test (TC-E2E-02). Each step of the test required careful sequencing — tokens had to be obtained before protected endpoints could be called, and the middleware session mock had to be applied at the fixture level to ensure log entries were accessible for assertions in Step 13. Database state was verified using db_session.expire_all() followed by direct ORM queries to confirm that each transition persisted correctly.

Day 5 – Final Validation and Regression Testing

The final development day concentrated on end-to-end test execution, regression verification, and sprint closure. Manual testing confirmed that the full completion workflow behaved correctly across all three user roles. All 27 test files were executed to confirm that no regressions were introduced, and the coverage report was regenerated to confirm the 99% overall figure.

The sprint concluded with all remaining work reaching zero, indicating that all committed features and validation tasks were successfully completed.

Trend Interpretation

The burndown trend for Week 3 shows a steady reduction in remaining work with no scope expansion or unexpected task additions.

Key observations from the sprint progression include:

- No scope creep occurred during development.
- Backend middleware implementation was completed before frontend integration.
- Bug fixes were resolved before end-to-end testing commenced.
- All test cases were passing before the sprint was declared complete.

The development sequence followed a logical progression:

1. Logging middleware implementation
2. Unit test development and admin endpoint
3. Frontend integration and bug resolution
4. End-to-end pipeline test development
5. Final regression validation and sprint closure

This structured approach minimized integration risk and ensured that the system maintained consistent business logic across both backend and frontend components.

Velocity Alignment

Sprint 1 average velocity: ~16 points
Sprint 2 average velocity: ~14 points
Sprint 3 Week 1 completed: 16 points
Sprint 3 Week 2 completed: 16 points
Sprint 3 Week 3 completed: 16 points

The higher point total during Sprint 3 Week 3 reflects the complexity of implementing a cross-cutting concern (logging middleware) that spans the entire application, combined with the effort required to build a comprehensive 14-step end-to-end validation test. Despite the higher complexity, the sprint demonstrates continued stable and predictable delivery, indicating accurate estimation practices and effective sprint planning. All committed work was completed within the planned five-day development window.

8. Backlog Grooming

Sprint Backlog

All Sprint 3 issues completed.

Product Backlog Changes

During Sprint 3 Weeks 1 and 2, the product backlog was groomed in preparation for Sprint 4. Four new user stories (US-16 through US-19) were identified, prioritized, and added to the backlog to extend the MicroMatch platform beyond its core workflow with user awareness, communication, analytics, and profile enhancement capabilities.

New Stories Added to The Product Backlog (Sprint 4 Candidates):

ID	User Story	Priority	Story Points	Sprint	Rationale
US-16	Notifications	Medium	5	Sprint 4	Notify users of key events including application decisions, deliverable reviews, and project completion (FR-16).
US-17	Messaging	Medium	8	Sprint 4	Allow users to send and receive messages within the context of a project (FR-17).
US-18	Analytics	Low	5	Sprint 4	Provide users with basic analytics including counts of applications and completed projects (FR-18).
US-19	Profile Enhancements	Low	3	Sprint 4	Allow profiles to display skills, completed projects, and ratings (FR-19).

No existing stories (US-1 through US-15) were reprioritized or removed. The backlog remains stable and fully aligned with the original project roadmap. No changes were made to the sprint backlog or product backlog during Sprint 3 Week 3; specifically, sprint execution followed the planned structure defined in Weeks 1 and 2

Sprint 4 will begin with:

- Notifications (US-16) — Medium priority, 5 SP

- Messaging (US-17) — Medium priority, 8 SP
- Analytics (US-18) — Low priority, 5 SP
- Profile Enhancements (US-19) — Low priority, 3 SP

Additionally, the following technical improvements identified during the Sprint 3 retrospective are queued as Sprint 4 backlog items:

- Increase branch coverage to 100% for app/core/dependencies.py, app/routers/projects.py, and app/routers/organization_profile.py.
- Set up GitHub Actions CI pipeline for automated test runs on every push.
- Introduce Vitest / React Testing Library frontend unit tests for key components.
- Implement pagination on the GET /admin/logs endpoint.

Backlog remains stable and aligned.

9. Final Sprint 3 Outcome

Sprint 3 delivered:

- Deliverable submission and review workflow
- Project completion endpoint with RBAC enforcement
- Post-completion feedback with completion guard
- Admin moderation and system log access
- System-wide logging middleware (FR-15)
- 14-step end-to-end completion workflow validation
- 99% test coverage across 1,350 statements
- 100% passing test suite across 27 test files
- Zero unresolved defects

Sprint Goal: **Successfully Achieved**

10. Sprint Retrospective

The Sprint 3 Week 3 retrospective evaluates the development process, technical implementation, and collaboration outcomes during the sprint. I reflected on successes, challenges encountered, and actionable improvements that will strengthen future iterations of the project.

The retrospective discussion focused on three key questions:

1. What went well?
2. What did not go so well?
3. What can be improved in upcoming sprints?

10.1 What Went Well

Several aspects of the sprint were successful and contributed to steady development progress and overall sprint completion.

Test-Driven Development for the Logging Middleware

One of the most positive outcomes of the sprint was the adoption of a test-driven approach for the logging middleware. By writing the six unit tests before finalizing the middleware implementation, it was straightforward to verify each acceptance criterion incrementally. The test file effectively served as a living specification, ensuring that unauthenticated logging, authenticated user capture, error status logging, timestamp recording, and HTTP method recording were all validated before the feature was considered complete.

Comprehensive End-to-End Pipeline Test

The 14-step end-to-end completion test (TC-E2E-02) proved to be an extremely effective integration harness. By scripting the entire workflow, from registration through admin log verification, in a single automated test with database state checks at each step, the test surfaced integration issues that isolated unit tests would not have caught. The feedback guard logic and the admin log retrieval endpoint were both validated as a natural byproduct of the pipeline test, reducing the number of additional unit tests required.

Effective Middleware Session Mocking Pattern

The use of unittest.mock.patch to redirect the middleware's SessionLocal to the SQLite test database was an elegant solution to the challenge of testing middleware-initiated database writes without requiring a separate database instance. This pattern is reusable across any future middleware test and allowed the logging test suite to make direct ORM assertions on log entries without any additional infrastructure.

High Overall Test Coverage

The overall backend test coverage of 99% across 1,350 statements provides a strong safety net for any future refactoring or feature additions. Full 100% coverage of all models, schemas, and authentication logic means that data integrity and access control are thoroughly protected by the automated test suite.

Stable Sprint Scope

The sprint backlog remained stable throughout Week 3. No stories were added mid-sprint, and the two bug fixes that were identified were small enough to be resolved within the existing commitment without requiring rescoping. This maintained development momentum and allowed the sprint to close cleanly.

10.2 What Did Not Go So Well

Although the sprint was completed successfully, several challenges and inefficiencies were encountered during development.

Tailwind CSS Misconfiguration Diagnosis Time

The Tailwind CSS content path misconfiguration (BUG-S3-02) was a significant time sink. Because the development server did not emit a clear warning when utility classes were purged, the issue was not immediately obvious. Diagnosing the problem required comparing the production CSS bundle against expected utility class output, which took several hours of investigation before the root cause in tailwind.config.js was identified.

User's Name Persistence Gap

The user's name persistence bug (BUG-S3-03) was introduced because the User model and registration schema were extended separately without a synchronized checklist. In a multi-developer environment, this type of issue would typically be caught in code review, but as a solo project, it was not discovered until manual frontend testing surfaced the missing field. This highlighted a gap in the development workflow for model-schema changes.

Middleware Scope Estimation

Scope estimation for the logging middleware was slightly optimistic. While the middleware class itself was straightforward to implement, the total effort, including JWT parsing within the middleware context, wiring the admin endpoint, building the React log view, and writing

six comprehensive test cases with session mocking, collectively exceeded the initial time budget for the story. Future middleware feature estimates should account for integration complexity and test infrastructure setup separately from the core implementation work.

Frontend Testing Coverage

While backend coverage reached 99%, frontend testing remained entirely manual throughout Sprint 3. The React components — including the admin logs view, feedback form, and deliverables dashboard — were validated through manual interaction rather than automated UI tests. This creates a risk of undetected frontend regressions in future iterations.

10.3 What Can Be Done Better

Based on the challenges encountered during the sprint, I identified several concrete improvements that will help strengthen future development cycles.

Implement a Model-Schema Change Checklist

To prevent issues like BUG-S3-03, a formal checklist will be applied before committing any changes to SQLAlchemy models. The checklist will verify that:

- The corresponding Pydantic schema is updated.
- The registration or create endpoint includes the new field.
- Relevant test fixtures are updated to include the new field.
- The frontend rendering component reflects the new field.

This four-point check takes under two minutes and will prevent model-schema drift from reaching integration testing.

Add Frontend Smoke Tests for CSS Changes

To avoid the diagnosis overhead caused by BUG-S3-02, future frontend changes involving CSS configuration will be validated with a visual smoke test immediately after the change. This can be as simple as a manual screenshot comparison of the main pages before and after the configuration update, ensuring that styling regressions are caught before committing.

Close Remaining Coverage Gaps

The three files currently below 100% coverage each have identifiable, testable uncovered branches. Writing parametrized pytest tests for those edge cases will close the remaining 1% gap:

- app/core/dependencies.py (lines 34, 43): Add tests for malformed JWT edge cases.
- app/routers/projects.py (lines 78–85): Add test for administrative bulk-delete guard.
- app/routers/organization_profile.py (lines 28, 66, 86): Add tests for missing-profile scenarios.

Set Up Continuous Integration

Setting up a GitHub Actions workflow to run pytest and generate a coverage badge on every push would surface regressions before they accumulate. For a solo project, a simple workflow that runs pytest --cov and fails if coverage drops below 95% provides a low-maintenance, high-value safety net. This is identified as the highest-priority backlog action item for the next iteration.

Introduce Frontend Automated Testing

The absence of automated frontend tests represents a testing gap that will grow as the React codebase expands. Future iterations will introduce Vitest or React Testing Library unit tests for key components — particularly the feedback form, deliverables dashboard, and admin logs view — to reduce reliance on manual UI validation.

Improve Sprint Estimation Granularity for Cross-Cutting Concerns

The logging middleware experience revealed that cross-cutting concerns require a different estimation approach than feature endpoints. Future stories involving middleware, authentication changes, or schema migrations will be broken into sub-tasks covering implementation, test infrastructure setup, and UI integration separately, allowing more accurate point allocation and better visibility into progress.

10.4 Retrospective Summary

Sprint 3 Week 3 was successful and resulted in the completion of all committed user stories and technical tasks. The sprint demonstrated strong progress in logging infrastructure implementation, comprehensive end-to-end pipeline validation, and system-wide test coverage.

While some challenges occurred related to CSS misconfiguration diagnosis, model-schema synchronization, and middleware scope estimation, these issues were successfully resolved and clear strategies were identified to prevent recurrence in future iterations.

By implementing a model-schema change checklist, frontend CSS smoke tests, CI pipeline integration, and frontend automated testing, future development cycles will be more efficient, better instrumented, and less reliant on late-stage manual discovery. The MicroMatch platform has been delivered as a fully functional micro-internship marketplace with a robust, well-tested backend and a clear path toward production readiness.